Git Multi-Device Development Workflow Guide

Managing Development Across Laptop and Desktop PC

Date: November 16, 2025
Project: Pusti Happy Times - MERN & React Native
Author: Development Team Guide

🎯 The Golden Rule

> **"Commit often, push frequently, pull before you start working"**

This simple rule will save you from 99% of Git conflicts and confusion.

📋 Recommended Workflow

Setup (One-time)

On Your Laptop (Current Setup)

```
# 1. Check current status
git status
git branch  # See which branch you're on

# 2. Ensure all changes are committed
git add .
git commit -m "Latest work from laptop"

# 3. Push to remote (GitHub/GitLab)
git push origin main
```

On Your Desktop PC (First Time Setup)

```
# 1. Clone the repository
cd C:\Development  # or wherever you want
git clone https://github.com/rockrgomesai/pusti-happy-times.git
cd pusti-happy-times

# 2. Verify connection
git remote -v  # Should show your GitHub repo

# 3. Pull latest code
git pull origin main
```

🔄 Daily Workflow

SCENARIO 1: Starting Work on Desktop PC

```
# STEP 1: Always pull latest changes first
git pull origin main

# STEP 2: Check for conflicts (Git will tell you)
# If no conflicts, you're good to go!

# STEP 3: Work on your code (React Native development)
# ... make changes ...

# STEP 4: Commit your work (even if not finished)
git add .
git commit -m "WIP: Working on attendance screen"

# STEP 5: Push to remote
git push origin main
```

SCENARIO 2: Switching Back to Laptop

```
# STEP 1: Pull latest changes from desktop
git pull origin main

# STEP 2: Continue working
# ... make changes ...

# STEP 3: Commit and push
git add .
git commit -m "Fixed backend API route"
git push origin main
```

🎨 Visual Workflow

```
┌──────────────────────────────────────────────────────────┐
│                       GitHub (Remote)                    │
│               https://github.com/user/repo               │
└──────────────────────────────────────────────────────────┘
                     ↑                     ↑
                     | git push            | git push
                     |                     |
          ┌──────────┴──────────┐     ┌────┴────────────────┐
          │                     │     │                     │
          │   Laptop (Web)      │     │   Desktop (Mobile)  │
          │   - Next.js         │     │   - React Native    │
```

```
            │   - Backend          │     │   - Android SDK      │
            │                      │     │                      │
            └──────────────────────┘     └──────────────────────┘
                        │                      │
                        │ git pull             │ git pull
                        └──────────────────────┘
```

🚨 Handling Conflicts (Important!)

When You Forget to Pull First

```
# You make changes on laptop, try to push
git push origin main

# ERROR: Updates were rejected because the remote contains work...
# This means someone (you on desktop) pushed changes

# SOLUTION:
git pull origin main  # Pull the changes first

# If there are CONFLICTS, Git will show them
# Open the conflicted files, you'll see:

<<<<<<< HEAD
// Your laptop code
const API_URL = 'http://localhost:5000';
=======
// Your desktop code
const API_URL = 'http://192.168.1.100:5000';
>>>>>>> origin/main

# RESOLVE: Choose which code to keep or merge both
const API_URL = process.env.API_URL || 'http://localhost:5000';

# Then:
git add .
git commit -m "Resolved conflict in API_URL"
git push origin main
```

💡 Best Practices for Your Setup

1. Use Feature Branches (Recommended)

Instead of working directly on `main`, create branches:

```
# On Desktop (starting React Native work)
git checkout -b feature/mobile-attendance
```

```
# ... work on React Native ...
git add .
git commit -m "Add attendance screen"
git push origin feature/mobile-attendance

# On Laptop (working on backend)
git checkout -b feature/attendance-api
# ... work on backend API ...
git add .
git commit -m "Add attendance endpoints"
git push origin feature/attendance-api

# When both are done, merge on GitHub
# Then pull merged code on both machines
```

Benefits:

- ☑ No conflicts between laptop and desktop work
- ☑ Work on different features simultaneously
- ☑ Easy to review changes before merging
- ☑ Can discard branch if feature doesn't work out

2. Commit Messages Strategy

```
# Good commits (descriptive)
git commit -m "feat: Add geolocation service for attendance"
git commit -m "fix: Resolve distance calculation bug"
git commit -m "refactor: Extract API client to separate file"
git commit -m "WIP: Attendance screen UI (not finished)"

# Bad commits (avoid)
git commit -m "update"
git commit -m "fixes"
git commit -m "asdfjkl"
```

3. What to Commit vs Ignore

ALWAYS COMMIT:

- ☑ Source code (`.js`, `.ts`, `.tsx`, `.jsx`)
- ☑ Configuration files (`package.json`, `tsconfig.json`)
- ☑ Documentation (`.md` files)
- ☑ Environment templates (`.env.example`)

NEVER COMMIT:

- ❌ `node_modules/` (too large, auto-generated)
- ❌ `.env` files (contain secrets)

- ❌ Build outputs (`dist/`, `build/`, `.next/`)
- ❌ IDE files (`.vscode/`, `.idea/`)
- ❌ OS files (`.DS_Store`, `Thumbs.db`)
- ❌ Android build files (`android/app/build/`)

Your `.gitignore` should handle this automatically.

🗂 Recommended Folder Structure for Your Setup

Option 1: Separate Repositories (Recommended)

```
C:\Development\
├── pusti-ht-mern\              # Laptop - Web app (existing)
│   ├── backend/
│   ├── frontend/
│   └── .git/
│
└── pusti-ht-mobile\            # Desktop - React Native (new)
    ├── android/
    ├── ios/
    ├── src/
    └── .git/
```

Option 2: Monorepo with Separate Folders

```
C:\Development\pusti-happy-times\
├── backend/                     # Shared backend
├── web/                        # Next.js web (laptop)
├── mobile/                     # React Native (desktop)
└── .git/                       # Single Git repo
```

Recommendation: Option 1 because:

- React Native has HUGE `node_modules` and Android build files
- Separate repos = faster git operations
- Can work offline on one without affecting the other
- Cleaner separation of concerns

🛠 Practical Commands Cheat Sheet

Before Starting Work on Any Machine

```
git status                    # Check current state
git pull origin main          # Get latest changes
```

During Work

```
git add .                      # Stage all changes
git add src/screens/Attendance.tsx  # Stage specific file
git commit -m "Your message"  # Commit with message
git push origin main          # Upload to GitHub
```

Check What Changed

```
git status                     # See modified files
git diff                       # See exact changes
git log --oneline              # See commit history
```

Undo Mistakes (BEFORE Pushing)

```
git restore file.txt           # Discard changes to file
git reset HEAD~1               # Undo last commit (keep changes)
git reset --hard HEAD~1        # Undo last commit (discard changes)
```

Branch Management

```
git branch                     # List branches
git checkout -b feature-name  # Create and switch to new branch
git checkout main              # Switch back to main
git merge feature-name         # Merge branch into current
```

Emergency: Reset to Remote Version

```
git fetch origin               # Download latest
git reset --hard origin/main  # Reset to match remote (CAUTION!)
```

🎯 Your Specific Workflow

For Web Development (Laptop - Low Power)

```
# Morning routine
cd C:\tkg\pusti-ht-mern
git pull origin main
```

```
# Work on:
# - Backend API routes
# - Next.js frontend (light tasks)
# - Database schemas
# - Bug fixes

# Before closing laptop
git add .
git commit -m "Completed attendance API endpoints"
git push origin main
```

For React Native Development (Desktop - High Power)

```
# When switching to desktop
cd C:\Development\pusti-ht-mobile
git pull origin main

# Work on:
# - React Native UI components
# - Android builds
# - Native modules
# - Testing on emulator/device

# End of day
git add .
git commit -m "Completed attendance screen with GPS"
git push origin main
```

🚀 GitHub Desktop (Visual Interface)

If Git commands feel overwhelming, use **GitHub Desktop**:

1. **Download**: https://desktop.github.com/
2. **Install on both laptop and desktop**
3. **Visual interface for**:
   - ☑ Pulling latest changes (sync button)
   - ☑ Committing changes (checkboxes + message)
   - ☑ Pushing to remote (publish/push button)
   - ☑ Resolving conflicts (visual diff)
   - ☑ Switching branches (dropdown)

Much easier than command line for beginners!

⚠ Common Pitfalls to Avoid

1. ❌ **Forgetting to pull before starting work**

    - Always `git pull` first thing!

2. ❌ **Committing sensitive data**
    - Never commit `.env` files with passwords
    - Use `.env.example` as template
3. ❌ **Working on same file simultaneously**
    - Use branches for different features
    - Communicate which files you're editing
4. ❌ **Not committing frequently enough**
    - Commit after each logical change
    - Don't wait till end of day
5. ❌ **Pushing broken code**
    - Test before committing
    - Use `git commit -m "WIP: ..."` for incomplete work

📚 Learning Resources

- **Interactive Tutorial**: https://learngitbranching.js.org/
- **Visual Guide**: https://marklodato.github.io/visual-git-guide/index-en.html
- **Git Cheat Sheet**: https://education.github.com/git-cheat-sheet-education.pdf
- **Official Documentation**: https://git-scm.com/doc

🎬 Action Plan for Setup

Step 1: Setup Today (30 minutes)

```
# On Laptop (NOW)
cd C:\tkg\pusti-ht-mern
git add .
git commit -m "Current state before desktop setup"
git push origin main

# On Desktop (WHEN YOU SWITCH)
cd C:\Development
git clone https://github.com/rockrgomesai/pusti-happy-times.git
cd pusti-happy-times
git pull origin main
```

Step 2: Daily Workflow (3 minutes per day)

```
# Starting work
git pull origin main

# Ending work
git add .
git commit -m "What you did today"
git push origin main
```

Step 3: When Creating React Native App

Create **separate repository** for mobile:

```
# On Desktop
cd C:\Development
npx react-native init PustiMobile
cd PustiMobile

# Initialize Git
git init
git add .
git commit -m "Initial React Native setup"

# Create GitHub repo and push
git remote add origin https://github.com/rockrgomesai/pusti-happy-times-mobile.git
git push -u origin main
```

☑ Quick Reference Card

Every Day Routine

When	Command	Why
Start work	`git pull origin main`	Get latest changes
After changes	`git add .`	Stage all files
After changes	`git commit -m "message"`	Save changes locally
End of work	`git push origin main`	Upload to GitHub

When Things Go Wrong

Problem	Solution
Forgot to pull	`git pull origin main` then resolve conflicts
Made mistake	`git restore file.txt` (before commit)
Wrong commit	`git reset HEAD~1` (before push)

Problem	Solution
Total mess	`git reset --hard origin/main` (CAREFUL!)

Feature Branch Workflow

```
# Start new feature
git checkout -b feature/my-feature

# Work and commit
git add .
git commit -m "Add feature"

# Push feature branch
git push origin feature/my-feature

# Merge on GitHub, then locally:
git checkout main
git pull origin main
git branch -d feature/my-feature  # Delete local branch
```

🎓 Advanced Tips

Stashing Changes (When You Need to Switch Quickly)

```
# You're working but need to pull/switch branches
git stash                    # Save work temporarily
git pull origin main         # Update code
git stash pop                # Restore your work
```

View What Others Changed

```
git log --oneline --graph --all   # Visual commit history
git show commit-hash              # See specific commit changes
```

Tagging Releases

```
# Mark important versions
git tag -a v1.0.0 -m "First production release"
git push origin v1.0.0
```

🆘 Emergency Procedures

Scenario: Pushed Wrong Code to Main

```
# Find the good commit
git log --oneline

# Reset to that commit (replace abc123 with actual hash)
git reset --hard abc123

# Force push (CAREFUL - only if working alone!)
git push --force origin main
```

Scenario: Need to Recover Deleted Files

```
# File deleted but not committed
git restore file.txt

# File deleted and committed
git log --all --full-history -- file.txt  # Find commit
git checkout commit-hash -- file.txt       # Restore it
```

Scenario: Accidentally Committed Secrets

```
# Remove from latest commit
git rm --cached .env
git commit --amend -m "Remove .env file"
git push --force origin main

# Better: Use git-filter-repo to remove from history
# Or rotate all secrets immediately!
```

📞 Support & Help

When Stuck:

1. **Read the error message** - Git is usually clear about what's wrong
2. **Check git status** - Shows current state and suggestions
3. **Use GitHub Desktop** - Visual interface helps understand state
4. **Ask for help** - Provide the error message and what you tried

Useful Commands for Troubleshooting:

```
git status              # What's changed?
git log --oneline -10   # Recent commits
git remote -v           # Which remote am I connected to?
git branch -a           # All branches (local and remote)
git diff                # What exactly changed in files?
```

📝 Summary: Your Action Checklist

- ☐ Commit and push all current laptop work
- ☐ Clone repository on desktop PC
- ☐ Install GitHub Desktop on both machines (optional)
- ☐ Practice: Make small change on laptop, push, pull on desktop
- ☐ Setup `.gitignore` for React Native project (when created)
- ☐ Create separate repo for mobile app
- ☐ Follow daily routine: Pull → Work → Commit → Push

🎯 Key Takeaways

1. **Always pull before starting work** - This prevents most conflicts
2. **Commit frequently** - Small, logical commits are better
3. **Use descriptive messages** - Your future self will thank you
4. **Feature branches for big changes** - Keeps main stable
5. **Don't panic on conflicts** - They're normal and fixable
6. **GitHub Desktop exists** - Use GUI if commands are confusing
7. **Separate repos for separate concerns** - Mobile and web apart

---

Remember: Git is a safety net, not a burden. Once you get the routine down (pull → work → commit → push), it becomes second nature!

Happy coding! 🚀

Last updated: November 16, 2025
Project: Pusti Happy Times Development